

Akka vs. Orkes (Conductor)

A comparison for teams building agentic AI

July 2026

Orkes Conductor orchestrates your agents; Akka runs your whole agentic system — and guarantees it. Orkes Conductor is a durable workflow and agent-orchestration engine from the Netflix Conductor lineage — proven, event-driven, and one layer. It coordinates steps and tools well, but you bring the agents, durable memory, streaming, governance, and the application platform around it. Akka delivers them as one platform, at higher guarantees — and Akka Specify can deliver and run the whole system for you as a guaranteed outcome.

99.9999%

AKKA AVAILABILITY SLA

Weeks

AKKA SPECIFY DELIVERY

Fixed

ONE ALL-IN PRICE

90%

LESS INFRASTRUCTURE

DIMENSION	ORKES (CONDUCTOR)	AKKA
What it is	A durable workflow / agent-orchestration engine (managed Netflix Conductor)	A full-stack agentic systems platform
Scope	Orchestration core; agents, durable memory, streaming, and AI governance are sourced, integrated, and operated by the customer	Orchestration, agents, memory, streaming, APIs, observability, and governance on one runtime
How you reach production	Self-integrate the full stack around orchestration, or contract a labor-led integrator	Build with spec-driven development, or Akka Specify delivers and runs it
Commercial model	Managed-cluster / consumption pricing + separate infra for every other layer + build-and-operate labor	One fixed price — platform, infrastructure, tokens, training, delivery, and operations
Outcome guarantee	Effort only; the outcome is not guaranteed	The delivered outcome is guaranteed
Availability SLA	Up to 99.99% (multi-region clusters); 99.9% on managed clusters	99.9999% — entire platform, backed by indemnities
RTO / RPO	Not published as a numeric guarantee	Sub-1-minute RTO; zero-byte RPO; active-active HA/DR
AI agents	Agents implemented as durable workflows; Agentspan OSS agent runtime, MCP/API Gateway, multi-LLM orchestration	Native agents with tools, handoffs, guardrails, interaction logging
Memory	None native — integrates external vector databases	Durable in-memory, 4ms reads / sub-10ms writes
State / durability	External datastores (Redis default + Elasticsearch; pluggable Postgres/MySQL/Cassandra), customer-operated	Durable sharded in-memory state, replayable from its event journal
Real-time streaming	Change-data-capture events to external brokers (Kafka, SQS, Pub/Sub); worker-polling task queues	Built in, backpressured, petabyte-scale, no external broker
Governance / EU AI Act	RBAC, audit logs, secrets — security/ops controls; no AI-policy enforcement, explainability, or classification	Aspect-woven runtime enforcement + full pre-production governance
Certifications	SOC 2 Type II; HIPAA via isolated deployment	19 standards (SOC 2 II + public SOC 3, ISO 27001/42001, HIPAA, PCI DSS, GDPR, NIS2,

		DORA, EU AI Act, NIST AI RMF)
Cost model	Managed clusters / consumption; customer provisions and operates the surrounding stack	Shared compute; up to 90% lower infrastructure for the same workload
Improves after go-live	Not provided	Continuous evaluation, reinforcement learning, and distillation — up to 80% lower token cost over time
Business model	Venture-funded: \$60M Series B, April 2026 (AXA Venture Partners)	Profitable; Dell Technologies Capital largest shareholder, customer, and AI partner

1. Orchestration Is One Layer; Akka Is the Full Stack

Orkes Conductor solves durable orchestration of long-running workflows and agents, and it solves it well — that is its heritage from Netflix Conductor. But an agentic AI system needs far more than orchestration, and Orkes leaves the rest to you. In Conductor, an AI agent is implemented as a workflow that calls an LLM to decide its next step; the agents, durable memory, streaming tier, and AI-governance stack are integrated and operated around the orchestration core, and you own every failure across those seams.

Capability	Orkes (Conductor)	Akka
Workflow / agent orchestration	Yes — durable, proven	Yes
Native AI agents (tools, handoffs, streaming)	Agents are workflows; Agentspan + MCP Gateway	Built in
Durable memory	None native — external vector DB	Built in, 4ms / sub-10ms
Real-time streaming	CDC to external brokers; no streaming engine	Built in, backpressured, petabyte-scale
HTTP / gRPC API layer	Gateway exposes workflows as endpoints	Built in
Governance / policy enforcement	RBAC, audit logs — no AI-policy enforcement	Inline, runtime-embedded
Pre-production governance	None	Classification, sign-offs, sealed posture

2. Two Ways to a Production System: Build It, or Have It Delivered

Getting from Orkes to a production agentic *system* means integrating agents, durable memory, streaming, and governance around the orchestration core — and then operating all of it. You do that yourself, or you contract a labor-led integrator to assemble it. Akka offers a second path entirely: **Akka Specify** takes your specifications and delivers and operates the whole governed system for you as a guaranteed outcome — in weeks, for one fixed price.

	With Orkes	With Akka Specify
Model	Self-integrate the full stack around orchestration, or a labor-led integration engagement	You provide the specifications
Who does the work	Your engineers, or forward-deployed / staff-augmentation contractors	Akka generates, governs, delivers, and runs the system
Timeline	Months to a full system, by construction	Weeks, not quarters
Billing	Managed-cluster / consumption pricing + separate infra + build-and-operate labor	One fixed price
Afterward	You own and operate every layer and every seam	Kept up, safe, and improving — operated by Akka
Guarantee	Effort is guaranteed; the outcome is not	The delivered outcome is guaranteed

Orkes gives you a proven, event-driven orchestration component to build around. Akka Specify hands you the finished, governed, running system — and keeps it available, safe, and improving under one agreement.

3. Availability and Disaster Recovery

Orkes Cloud publishes an availability SLA of [up to 99.99%](#) on multi-region clusters (99.9% on managed clusters), with service credits on confirmed breaches. That is real. Two gaps matter for mission-critical agentic systems: the SLA covers the orchestration platform, and

Orkes does not publish a numeric RTO/RPO guarantee. State lives in external datastores (Redis by default, Elasticsearch for indexing, pluggable Postgres/MySQL/Cassandra) that the customer provisions and operates, each under its own availability terms.

Metric	Orkes (Conductor)	Akka
Availability SLA	Up to 99.99% (multi-region); 99.9% managed clusters	99.9999%
Allowed downtime / year	~52 minutes (99.99%)	~31 seconds
RTO	Not published as a numeric guarantee	Sub-1 minute
RPO	Not published as a numeric guarantee	Zero byte
SLA scope	The orchestration platform	The entire platform
Who operates it	The customer (the assembled stack)	Akka SREs, 24/7

Orkes's SLA covers the orchestration platform. The memory, streaming, and governance services a customer integrates operate under their own SLAs — or none. When Akka delivers through Akka Specify, that entire running system is operated by Akka under one SLA.

4. Cheaper to Operate — and It Keeps Getting Cheaper

AI systems built with Akka are up to **90% cheaper to operate** than Python-based systems — a function of the infrastructure required for the same agentic transaction volume, not list price. With Orkes you run the orchestration engine plus the separate datastores, brokers, memory layer, and governance tooling around it, and you provision and operate each.

Akka runs all of it on one shared-compute runtime. The efficiency comes from actor concurrency (~10T tokens/core/year vs ~2T; ~80% less compute than Python frameworks; Manulife: up to 300% more concurrency, 30–50% faster processing after porting from Python), shared compute, and micro-checkpointing.

The delivered outcome also **compounds**: Akka Verify's continuous evaluation, reinforcement learning, and distillation cut token cost up to 80% over time. The spend is predictable — **one fixed price**, not consumption pricing that moves with load plus separate build-and-operate labor.

5. Governance and the EU AI Act

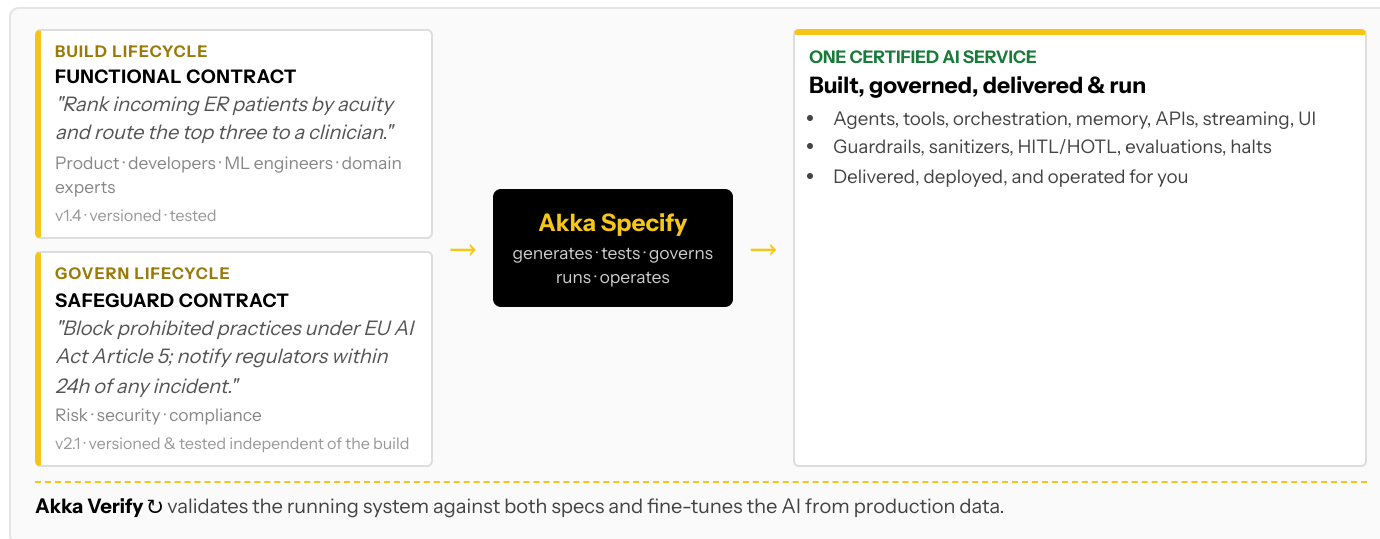
Orkes provides SOC 2 Type II and HIPAA (via isolated-cloud deployment), plus role-based access control, secret management, and audit logs — infrastructure-security and operational controls. It publishes no AI-governance capability: no real-time AI-policy enforcement, no decision explainability, no human pause/override of a running agent as a platform primitive, no immutable interaction ledger, no pre-deployment classification, and no sealed audit artifact.

Violation	Maximum Fine
Prohibited AI practices (Art. 5)	EUR 35M or 7% global turnover
High-risk obligations (Art. 9-15)	EUR 15M or 3% global turnover

High-risk AI carries a 10-year logging-retention obligation (Art. 72). **How Akka governs**: at the runtime — inline guardrails, policies, LLMs-as-a-judge, and sanitizers; hash-chained immutable evidence; HITL/HOTL control; classification against 186 regulations and 877 controls before a system ships; multi-persona sign-offs; a sealed Governance Posture Package; and Akka Verify proving conformance from the running system. Governance Orkes would have to bolt on, Akka enforces inline — and delivers it already wired when the system is built through Akka Specify.

6. One Certified System — Built, Governed, Delivered, and Run

Building on Orkes means engineers defining and operating durable workflows; there is no built-in governance lifecycle and no path for a product manager, domain expert, or risk officer to contribute a governed contract. Akka runs two independent lifecycles on one platform via **Akka Specify**:



Akka generates, tests, governs, **runs, and operates** one certified service from both specs, which are versioned and tested independently by different audiences. Through Akka Specify, that certified system is delivered and operated as a guaranteed outcome — a delivery model and an independent governance lifecycle Orkes has no equivalent for.

7. Real-Time Streaming at Petabyte Scale

Orkes Conductor streams workflow state changes to external brokers via change data capture (Kafka, SQS, Pub/Sub, NATS) and runs on worker-polling task queues; it has no built-in streaming engine, so real-time pipelines and backpressure are provisioned on infrastructure you operate. Akka's streaming is built into the runtime — continuous, backpressured, **petabyte-scale, in-memory**, with no external broker — powering both agent feedback loops and high-throughput data processing (the engine behind Tubi's real-time hyper-personalization at 5 billion tokens per second).

8. For the Buyer: Maturity, Risk, and Accountability

Orkes carries a genuine maturity advantage in orchestration: Conductor was open-sourced at Netflix in 2016, has 13,000+ GitHub stars, and runs in production at large enterprises — Orkes maintains the project after Netflix discontinued support in December 2023. Where the lines fall:

Buyer concern	Orkes (Conductor)	Akka
Orchestration maturity	Strong — Netflix lineage, proven OSS, large deployments	Durable execution substrate; 18 years, 100,000+ deployments
Certifications & audits	SOC 2 Type II; HIPAA via isolated deployment	19 standards — SOC 2 II + public SOC 3, ISO 27001/42001, HIPAA, PCI DSS, GDPR, NIS2, DORA, EU AI Act, NIST AI RMF — plus annual pen tests, SBOMs, 40+ policies (trust.akka.io)
Delivery & outcome	Self-integrate the full stack, or a time-and-materials integrator; effort is guaranteed, the outcome is not	Akka Specify delivers and operates the system for one fixed price; the outcome is guaranteed
Scope of accountability	The orchestration platform; you integrate and operate the rest	One platform, one SLA, 24/7 SRE — Akka owns the running system
Risk transfer	Service credits on confirmed SLA breach	Availability and data-integrity guarantees backed by contractual indemnities
Funding model	Venture-funded: \$60M Series B, April 2026 (AXA Venture Partners); prior \$20M Series A (2024); scaling on investor capital	Profitable and self-funding; Dell Technologies Capital is largest shareholder, a customer, and an AI partner
Budget predictability	Consumption / managed-cluster pricing plus the surrounding stack you operate	One fixed price finance can forecast

Orkes's orchestration heritage is real and durable — but Akka funds its roadmap and support from profit, not the next round, so the platform you standardize on does not depend on a venture timeline. The decision is scope and accountability: Orkes gives you a proven, event-driven orchestration engine to build a platform around; Akka gives you the platform — and, through Akka Specify, will deliver and run the whole system for you.

Customers Running Agentic and Real-Time Systems on Akka

Manulife 2,000 developers across 100 projects on one governed platform	Tubi 5B tok/s real-time hyper-personalization engine	Swiggy 71ms order-assignment AI, ~50% latency reduction	John Deere 1,000+ tractor sensors turned into real-time insight	Verizon 750% order-processing capacity gain; 6s → 2.4s response
--	--	---	---	---

Common Questions

We don't have a team to build this — what are our options?

Two. Build it on the platform with spec-driven development, or have Akka Specify deliver and operate it for you. You provide plain-language specifications; Akka generates, governs, delivers, and runs the whole system — agents, memory, streaming, APIs, orchestration, and governance — as a guaranteed outcome, for one fixed price. With Orkes you self-integrate that stack around the orchestration core or contract a labor-led integrator you then own.

How is Akka Specify different from hiring an integrator to build our agentic system on Orkes/Conductor?

An integration engagement sells effort — time-and-materials over months — and hands back a system you own and operate; the outcome is not guaranteed. Akka Specify sells the outcome: a governed system delivered in weeks for one fixed price, then kept up, safe, and improving by Akka. You own the specifications, not the operational burden.

We already run Orkes Conductor in production. Why add Akka?

Orkes added agents, Agentspan, and an MCP gateway. Doesn't that make it an agentic platform?

Those are real additions, but in Orkes an agent is a durable workflow that calls an LLM, with tools exposed through the gateway. The durable memory, streaming, governance, and application runtime around it are still yours to assemble. Akka is built for agentic AI end to end — native agents, built-in memory, embedded governance, and a full-stack runtime.

Can we add governance on top of Orkes?

You can add log-analysis and access controls, but the EU AI Act expects enforcement inline to the runtime: immutable records witnessed as they happen, human override on running processes, and authorization captured at execution time. RBAC and audit logs do not gate a deployment or classify a system before it ships. Akka embeds all of this and covers pre-deployment governance.

Orkes raised \$60M and is built on proven Netflix technology. Isn't that lower risk?

Orkes is strong for durable orchestration — that is its Netflix Conductor heritage. If you are moving into agentic AI you also need native agents, durable memory, real-time streaming, and runtime governance, which Orkes leaves you to source and operate. Akka can run alongside existing Conductor workflows while you build the agentic layer on one platform.

Sources

Orkes Series B: \$60M Series B, April 23, 2026, led by AXA Venture Partners (businesswire.com/news/home/20260423550324; finsmes.com; thesaasnews.com); prior \$20M Series A (2024), \$9.3M seed (2022)

Netflix lineage: orkes.io/what-is-conductor;

techcrunch.com/2023/12/13/orkes-forks-conductor — Conductor open-sourced at Netflix 2016; Orkes founded 2021 by Conductor creators; Netflix dropped support Dec 2023, Orkes maintains the fork

Orkes Cloud SLA: orkes.io/cloud-service-level-agreement; orkes.io/cloud — up to 99.99% (multi-region); 99.9% managed clusters; service credits on confirmed breach

Orkes AI / agents: orkes.io/content/ai-orchestration;

orkes.io/content/developer-guides/mcp-api-gateway;

playground.orkes.io/agentspan — agentic workflows, Agentspan runtime, MCP/API Gateway, multi-LLM, vector DB integration, HITL approvals

Orkes persistence / streaming: github.com/orkes-io/conductor-oss;

orkes.io/content/conductor-architecture — Redis default + Elasticsearch; pluggable Postgres/MySQL/Cassandra; CDC events to Kafka/SQS/Pub-Sub/NATS; worker-polling task queues

Conductor's orchestration maturity is real and an advantage in that layer. But strong funding and a proven orchestration core do not add the missing agentic layers, and Orkes is scaling on investor capital toward profitability. Akka funds its roadmap from profit, owns one SLA across the whole system, and backs it with indemnities.

Orkes security: orkes.io/blog/orkes-is-now-soc-2-type-2-compliant;

orkes.io/security — SOC 2 Type II; HIPAA via isolated deployment; RBAC, audit logs, secrets

Akka Specify (spec-driven delivery): akka.io / akka.io/llms.txt — you provide

the specifications; Akka generates, tests, governs, delivers, and operates the system as a guaranteed outcome; one fixed price covering platform,

infrastructure, tokens, training, delivery, and operations; delivered in weeks; continuous improvement via reinforcement learning and distillation (up to 80% lower token cost, higher accuracy)

Akka trust center: trust.akka.io — 19 compliance standards; SOC 2 II + public SOC 3; annual pen tests, SBOMs, 40+ policies

Akka performance: akka.io/blog/go-slow-to-go-fast — Manulife up to 300% more concurrency, 30–50% faster; ~10T vs ~2T tokens/core; ~80% less compute than Python

Akka platform: 99.9999% availability, active-active HA/DR, sub-1 min RTO, zero byte RPO (contractual indemnities); 186 regulations / 877 controls / 288 with financial penalty; 100,000+ deployments / 18 years; profitable; Dell Technologies Capital